

TODO BEFORE SUBMISSION: replace every [>>> ... <<<] marker below with a real screenshot (taken manually, Cmd+Shift+4) or the hand-drawn sketch photo.

Task A: Real Website Analysis

Website analyzed: www2.hm.com – «New In» section (H&M US, women's clothing)

Meta information

The page uses `<!DOCTYPE html>` and `lang="en-US"`. The `<head>` section contains `<meta charset="utf-8">`, `<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">`, and a `<title>` tag (“New In | H&M US”).

I also found three additional meta tags, all Open Graph tags used to control how the page previews when shared on social media: `og:title`, `og:site_name`, and `og:type`.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 1 – doctype, lang, charset, viewport and title, read directly from the live page. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 2 – the three Open Graph meta tags (`og:title`, `og:site_name`, `og:type`). <<<]

Semantic Structure

Overall H&M uses a lot of semantic markup: 1 `<header>`, 7 `<nav>`, 1 `<main>`, 3 `<section>`, as many as 60 `<article>` elements (every product card on the page is its own `<article>`), and 1 `<footer>`.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 3 – element counts, taken by running `document.querySelectorAll(...).length` on the live page. <<<]

However, I found a clear structural inconsistency in the footer: the same three link lists (the “Women/Men/Kids...” menu, the “Career at H&M...” menu, and the “Customer Service...” menu) exist twice – once inside a real `<nav>`, and a second time with identical content inside a plain `<div>` (most likely one copy is meant for mobile and the other for desktop). So this is really a navigation block, but the tag isn't semantic – the semantics get lost specifically in that second, duplicate copy.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 4 – the same three footer link lists found twice: inside `<nav>` and again inside a plain `<div>`. <<<]

Forms and Tables

The page has exactly one `<form>` – the filter/sort widget – and it does have a genuine `<fieldset>` with a `<legend>` (“Filter”). But the actual control is a `<button type="button">` that opens a JS drawer, not a native `<input>/<select>`. It works for the user, but in terms of “clean” form semantics there is no real HTML selection element there – everything runs on a JS widget.

There isn't a single `<table>` anywhere on the page. So if H&M had tabular data – a size chart, for example – they would most likely lay it out with div grids instead of an actual `table/thead/tbody` structure.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 5 – the page's only `<form>` (`fieldset + legend + button`), and confirmation that `<table>` count is 0. <<<]

Hand-drawn Sketch

The sketch below shows the page's nesting: `<html>` > `<header>` (logo + `<nav>` main menu) > `<main>` > `<section>` (“New In” banner) > `<section>` > several `<article>` cards (I drew 4 of the 60 product cards as an example) > `<footer>` > three `<nav>` menus (“Women”, “Career”, “Customer Service”) + one more `<section>` for the newsletter signup.

[>>> INSERT REAL SCREENSHOT HERE: Photo of my hand-drawn box diagram, signed and dated 13.09.2026. <<<]

Structural Errors and How I Avoided Them

Error 1: The footer navigation on H&M is duplicated – the same three link lists (“Women/Men/Kids...”, “Career at H&M...”, “Customer Service...”) appear once inside a `<nav>` and a second time, with identical content, inside a plain `<div>`, so

the second copy loses its semantic meaning. On my own pages, the navigation menu appears only once per page and is always wrapped in a real `<nav>` – see the `<nav>` element right after `<header>` on every page of my site, for example `order.html` (lines 19–31), `about.html` (lines 19–31), `catalog.html` (lines 19–31), and `contact.s.html` (lines 19–31). There is no duplicate copy anywhere.

Error 2: The sort/filter widget on H&M is built with a `<fieldset>+<legend>`, which is correct, but the actual control is a custom `<button>` instead of a native `<input>/<select>`, so standard form semantics are lost. On my own pages, every form control is a real HTML input element correctly linked to its `<label>` via matching `id/for`. For example, in `order.html` the size selector is a genuine `<select id="size">` paired with `<label for="size">` (lines 112–119), and the fulfillment method uses real `<input type="radio">` elements paired with their labels (`<input id="pickup">/<label for="pickup">` and `<input id="courier">/<label for="courier">`, lines 125–133). The same pattern (`<label for>` matching `<input id>`) is used for every field in the reservation form on `contacts.html` (lines 146–197), for instance `<label for="c-name">` with `<input id="c-name">`.

Error 3: H&M has no `<table>` element at all, even though tabular data like a size chart would logically fit one better than a div grid. On my own pages, wherever I have tabular data I used a proper `table/thead/tbody/tr/th/td` structure instead of a div grid: `about.html` has a `<table>` with `<caption>`, `<thead>` and `<tbody>` in the “Key Information” block (lines 100–115+, inside `<section id="kontakty-tablitsa">`), and `catalog.html` has the same real table structure in the “Price Examples” block (lines 81–95+, inside `<section id="tseny">`).

Additional Example: `gaissina.com`

I also checked a second real site, `gaissina.com` (a Kazakhstani women's clothing store, built with Angular), to compare its structure with H&M and with my own pages. It uses `<!DOCTYPE html>`, `lang="ru"`, `charset UTF-8`, and a `<meta name="viewport">` tag, but it has no meta description tag at all and the `<title>` is just “Gaissina” with no further page context.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 6 – doctype/lang/charset/viewport/title, read from the live `gaissina.com` page. <<<]

Overall the page uses a fair amount of semantics: 3 `<header>`, 2 `<nav>`, 1 `<main>`, 8 `<section>`, 20 `<article>` and 1 `<footer>`, plus 2 `<h1>` elements.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 7 – semantic element counts, taken the same way as for H&M. <<<]

The page has exactly one `<form>` (a search widget: a button that opens a search field). There is no `<label>` element anywhere in it – it relies only on `aria-label` attributes instead of a real `<label>`, so there's no `label/for` pairing at all, not even a broken one. There are also 0 `<table>` elements on the page.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 8 – the page's only `<form>` (a search widget with no `<label>`), and confirmation that `<table>` count is 0. <<<]

I also found a structural inconsistency similar in spirit to H&M's footer problem: the footer's link columns (“Каталог”, “Покупателям”, “O Gaissina”, “Связаться”) use the CSS class `"navbar-nav"` on their `<ul>` elements, which looks like it should be navigation – but not a single one of these lists is actually wrapped in a `<nav>` element; they sit directly inside plain `<div class="footer-column">` containers. So the footer menus read as navigation visually and even by their class name, but carry no semantic `<nav>` tag at all – a different flavor of the same underlying problem I found on H&M (semantics lost in the footer navigation), and something I avoided on my own pages, where the `<nav>` after `<header>` is the only navigation block and it is always a real `<nav>` (`order.html/about.html/catalog.html/contacts.html`, lines 19–31).

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 9 – the footer's link lists use a `"navbar-nav"` class but are not wrapped in a real `<nav>` element. <<<]

My Own DevTools Screenshots (`gaissina.com`)

For completeness, here are my own screenshots of `gaissina.com` taken directly from

m Chrome DevTools (Elements panel) on a mobile viewport, showing the homepage, the product list, and the footer with its link columns discussed above.

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 10 – homepage in DevTools, mobile viewport, Elements panel open on <body>. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 11 – product list (Брюки, Лоферы) with the DOM tree visible on the right. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 12 – footer with "Каталог" / "Покупателям" / "O Gaissina" / "Связаться" columns. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 13 – homepage banner ("NEW COLLECTION") with DevTools Elements panel. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 14 – gaissina.com search form markup, read from DevTools (AI assistance panel): a <button> toggle plus an <input type="search"> inside a plain <div>, with no <label> element anywhere in the form. <<<]

[>>> INSERT REAL SCREENSHOT HERE: Screenshot 15 – gaissina.com footer link columns and the DOM tree in DevTools, mobile viewport, confirming the footer structure discussed above. <<<]

Task B: Written Part

What makes up a web page? It is essentially a standard HTML text file where the content is wrapped in tags—such as <p>, <h2>, <table>, and so on. If a site lacks CSS and JavaScript, the page consists solely of HTML—structure and content—without styling or interactive scripts. How does the browser transform this file into what appears on your screen? The browser downloads the HTML file as plain text and uses the tags to build a tree of nested elements; for instance, the <body> contains a <header>, and inside that <header> sits an <h1>. This tree is known as the DOM (Document Object Model). Since there is no CSS, the browser applies its own built-in default styles—which is why <h1> headings appear large and bold, links are blue and underlined, and list items feature bullet points. Finally, the browser renders this tree on the screen from top to bottom, following the order of elements in the code. Why does this matter? Tags like <table>, <section>, and <blockquote> convey meaning to browsers, search engines, and screen readers (used by the visually impaired), signaling that an element is a data table or a quotation; a <div>, however, carries no semantic meaning at all. It is merely a container—an empty box to the browser—so the data inside lacks semantic context. You should use semantic tags whenever possible, reserving <div> only for situations where no appropriate semantic tag exists.

Example one: In the "About" HTML file, line 102—using a table instead of a div for the grid layout. Here, we have parameter-value data: city (Astana), Instagram, link, average check, and a number. This is classic tabular data: rows and columns with a clear relationship between headers and values. A table signals to the browser and screen reader that a specific value corresponds to a specific parameter. If you had implemented this using divs, you could have achieved the same visual appearance via CSS, but the semantic connection between the cells would be lost; a screen reader would simply read it as a series of unrelated lines of text. Example two: In the "About" HTML file, compare the div on line 168 with the section elements on lines 56 and 100. Look at the container usage on line 168: the div wraps a single, minor technical note. It isn't a standalone page topic; it's just a small sidebar note. In contrast, the section elements on lines 56 and 100 wrap entire semantic blocks: "How it began" and "Get information." Each has its own H3 heading and represents a logically complete section of content. The rule for section is that it denotes a distinct topic or section with a heading. A div is used when you simply need a wrapper without semantic meaning—solely for technical grouping. Example three: In the "Order" HTML file, line 52—a blockquote combined with a <cite> element. This displays an actual quote from a customer

. The blockquote tells the browser and screen reader that these are someone else's words—not the page author's text—while the <cite> element inside indicates the source or attribution for the quote. This allows it to be indexed as a quotation, and the screen reader can announce to the user that a quote follows. The "Submit" button: what happens now? In the "Order" HTML file, line 76, we see form action with a hash (#) and the post method. The action attribute tells the browser where to send the data—usually a server address. Using a hash (#) for the action isn't a real address; it simply links back to the current page. So, when you click "submit order," the browser attempts to send the data; however, since there is no recipient—and the server doesn't exist yet (that comes later in the course)—nothing actually gets sent anywhere. The page simply reloads itself visually, and the entered data is neither saved nor published.